
Software Requirements Specification

for

Plasma: The Social Intelligence Layer for Steam

Version 1.0 approved

Prepared by:

Tayyab Khalid - 23L-0501
Muhammad Wahaj - 23L-0560
Muhammad Saad - 23L-0679
Muhammad Ibrahim Alam - 23L-3075



NATIONAL UNIVE
of Computer & Emerging !

Date of submission: March 11, 2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Product Scope.....	2
1.5 References	2
2. Overall Description	2
2.1 Product Perspective	2
2.2 Product Functions.....	3
2.3 User Classes and Characteristics	5
2.4 Operating Environment	5
2.5 Design and Implementation Constraints.....	6
2.6 User Documentation	6
2.7 Assumptions and Dependencies	6
3. External Interface Requirements	7
3.1 User Interfaces.....	7
3.2 Hardware Interfaces.....	7
3.3 Software Interfaces.....	7
3.4 Communications Interfaces	8
4. System Features	8
4.1 The Pulse (Unified Activity Feed).....	8
4.2 The Rally (Event Coordination Engine).....	9
4.3 The Omni-Library (Manual Game Tracking).....	9
4.4 The Prestige (Meta-Gamification).....	10
5. Other Nonfunctional Requirements.....	11
5.1 Performance Requirements.....	11
5.2 Safety Requirements.....	11
5.3 Security Requirements.....	11
5.4 Software Quality Attributes.....	12
5.5 Business Rules.....	12
6. Other Requirements	12
Appendix A: Glossary.....	14
Appendix B: To Be Determined List.....	14

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This document specifies the software requirements for Plasma, version 1.0. It completely describes the scope of the web-based product, detailing the frontend user interface, the backend data aggregation engine, and the external API integrations required to operate the platform. This SRS covers the entire system, encompassing all four core subsystems: **The Pulse** (social feed), **The Rally** (event scheduling), **The Omni-Library** (manual game tracking), and **The Prestige** (achievement gamification).

1.2 Document Conventions

This document strictly follows standardized typographical conventions to ensure clarity across all engineering and design teams:

- **Bold Text:** Aside from headings/sub-headings, used to indicate core platform subsystems and major features (e.g., **The Pulse**, **The Rally**). These are defined in Appendix A.
- **Italic Text:** Used to define specific user states, personas, or intent modes (e.g., *Comp Mode*, *Chill Mode*).
- **Requirement Tags:** All functional requirements are tagged with a unique identifier (e.g., REQ-1) for traceability.

Furthermore, it is assumed that priorities defined for higher-level system features are automatically inherited by their detailed, underlying functional requirements unless explicitly stated otherwise.

1.3 Intended Audience and Reading Suggestions

This document is intended for a diverse project team, specifically full-stack developers, project managers, UI/UX designers, and quality assurance testers. The rest of this SRS contains the overall product description, external interface requirements, and detailed system features.

We suggest the following reading sequences based on the reader's role:

- **Project Managers:** Review Section 1 (Introduction) and Section 2 (Overall Description) to understand the project's strategic scope, followed by Section 4 (System Features) for sprint and milestone planning.
- **Developers:** Read the entire document sequentially, focusing heavily on Section 3 (External Interfaces) for specific API integration details and Section 4 for detailed stimulus/response sequences.
- **UI/UX Designers:** Focus on Section 2.3 (User Classes), Section 3.1 (User Interfaces), and the functional requirements in Section 4 to guide wireframing, user flows, and prototyping.

1.4 Product Scope

Plasma is a web-based “Social Intelligence Layer” designed for PC gamers. Its primary purpose is to decouple the social experience of gaming from heavy, resource-intensive desktop launcher applications. By dynamically aggregating data from the Steam Web API and providing a manual entry system via IGDB API for non-Steam titles, Plasma provides a unified, platform-agnostic hub for gamers to track achievements, schedule cooperative sessions, and share milestones.

Plasma is designed to bridge the gap between “launching a game” and “living a gaming life.” While Steam dominates as a marketplace, the modern PC gamer’s library is fragmented across Battle.net, Riot, Epic, and standalone clients. Plasma acts as a specialized “Second Screen” web application that aggregates a user’s entire gaming identity, both automatic (Steam) and manual (everything else), into a rich, community-focused social hub.

This software aligns directly with the broader business strategy of capitalizing on industry “launcher fatigue”. Instead of competing to sell video games as a storefront, Plasma aims to be the premier digital “Third Place” for gamers. It is a lightweight, highly engaging utility network that solves the specific logistical (timezones, scheduling) and social frictions of adult gaming groups.

1.5 References

This SRS refers to the following external documents and technical standards:

- **Steam Web API Documentation:** Valve Corporation details the OpenID 2.0 authentication flow and REST API endpoints for user summaries, application tracking, and achievements. Available at: <https://steamcommunity.com/dev>
- **IGDB (Internet Game Database) API Documentation:** Twitch Developers. Web API utilized for fetching non-Steam game metadata for the Omni-Library. Available at: <https://api-docs.igdb.com/#getting-started>
- **IEEE Recommended Practice for SRS:** Karl E. Wiegers’ industry-standard template (1999) which dictates the structural formatting, sections, and flow of this specification document.

2. Overall Description

2.1 Product Perspective

Plasma is a new, self-contained software product designed to operate as a “Second Screen” web application. It is not a replacement for existing desktop game launchers (like Steam or Battle.net), but rather a supplementary social layer that sits above them.

The system acts as a centralized data aggregator and social hub. It interfaces directly with the Steam Web API to extract automated user data and relies on the IGDB API to fetch accurate metadata for manually tracked non-Steam titles. Because Plasma does not launch games or manage local game files, it remains a lightweight, cloud-based platform.

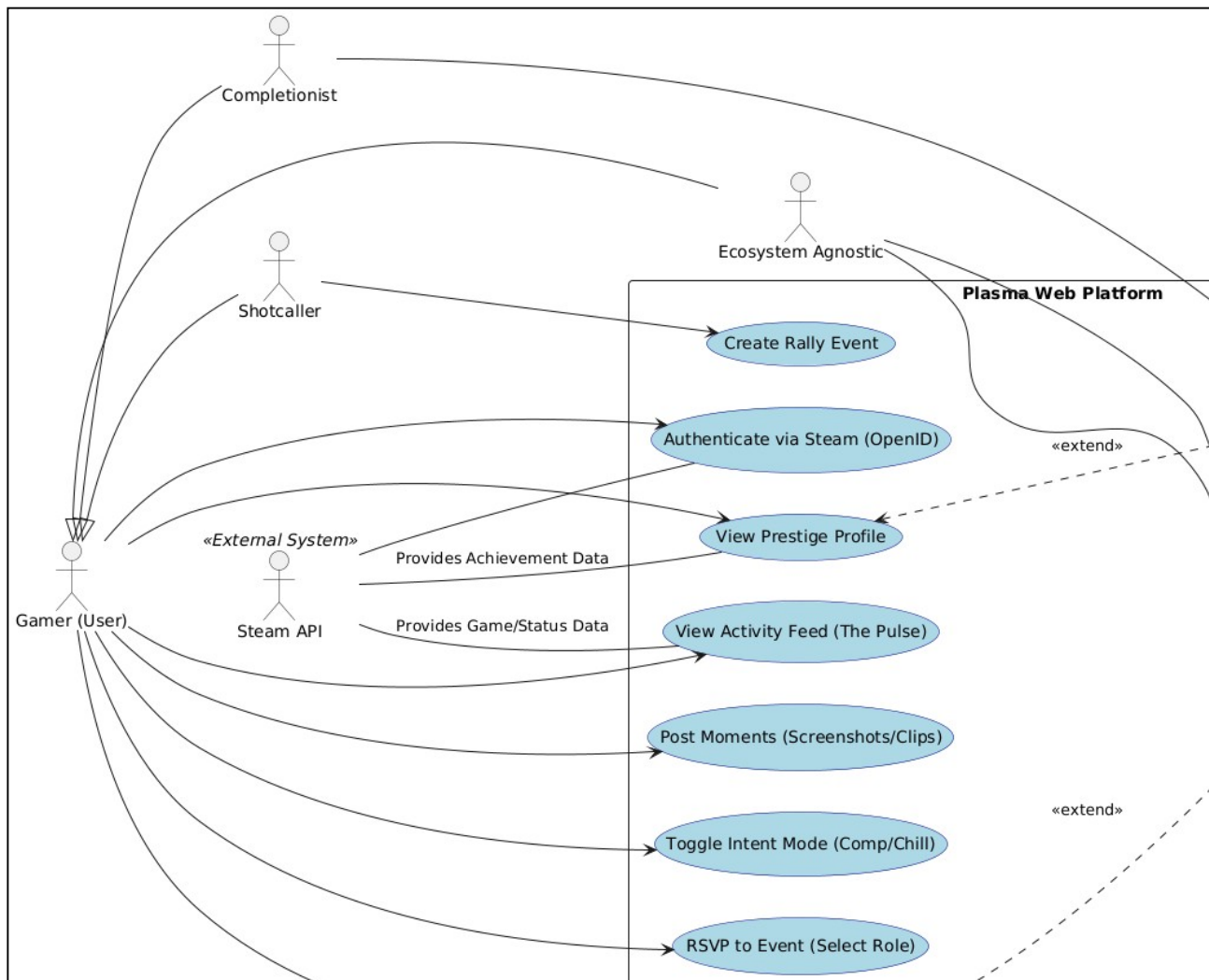
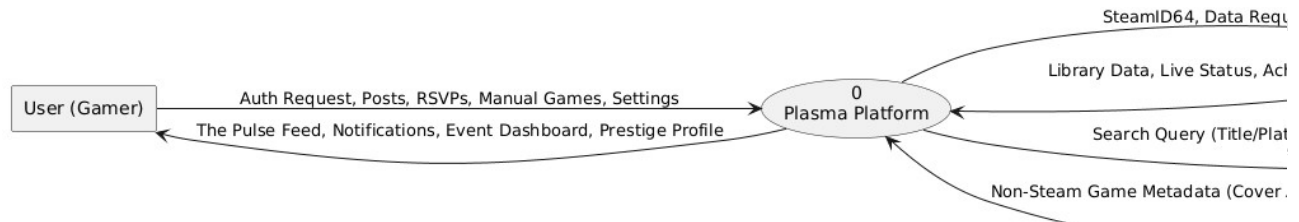
The major components of the overall system include the client-side Web Frontend (a responsive single-page application), the backend Identity Server (handling Steam OpenID 2.0 authentication),

the Aggregator Service (polling external APIs for live updates), and the primary relational database storing user social graphs and event schedules.

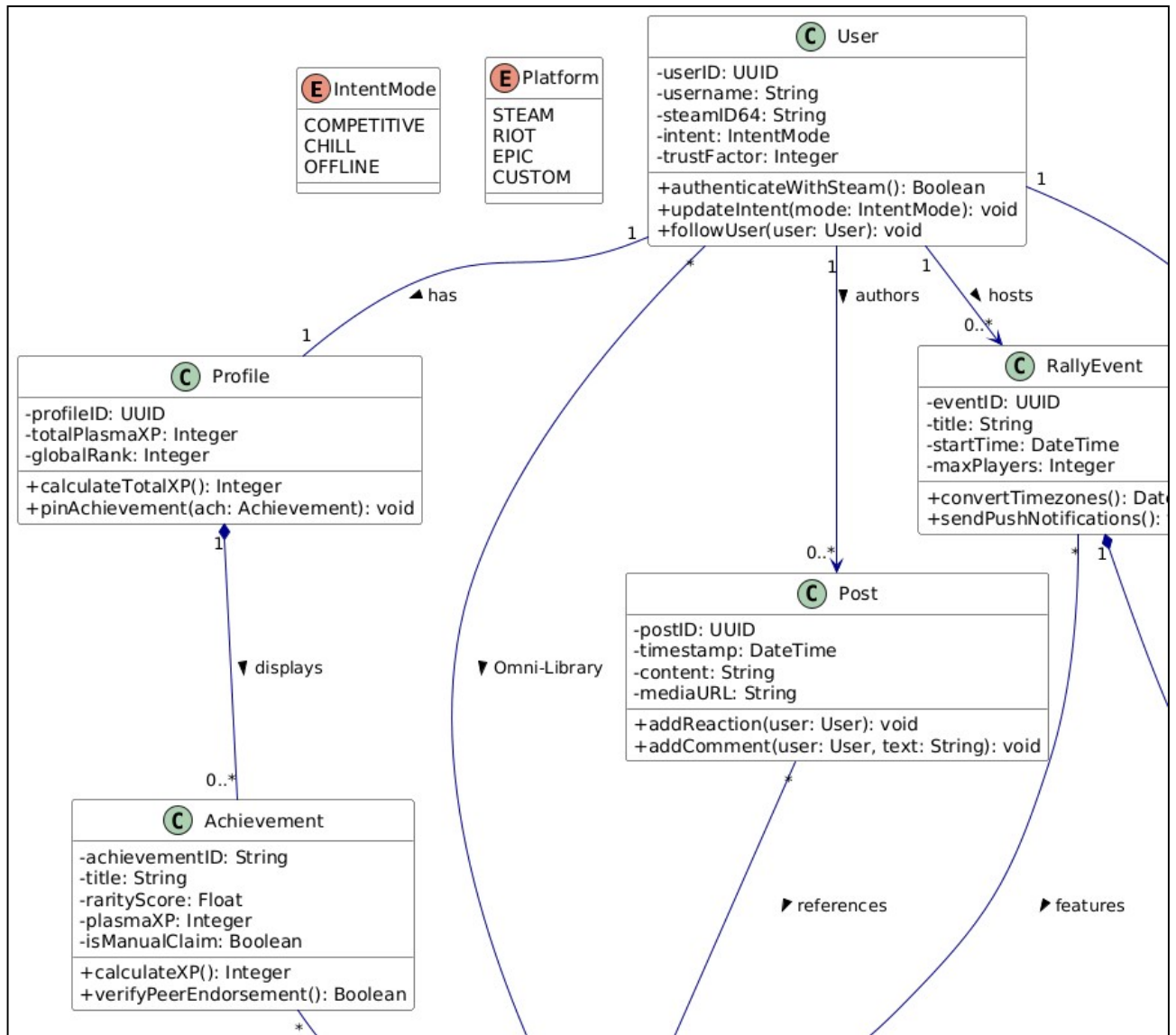
2.2 Product Functions

Plasma provides four major functional pillars designed to let the user socialize, schedule, and track their gaming life. These major functions are summarized below:

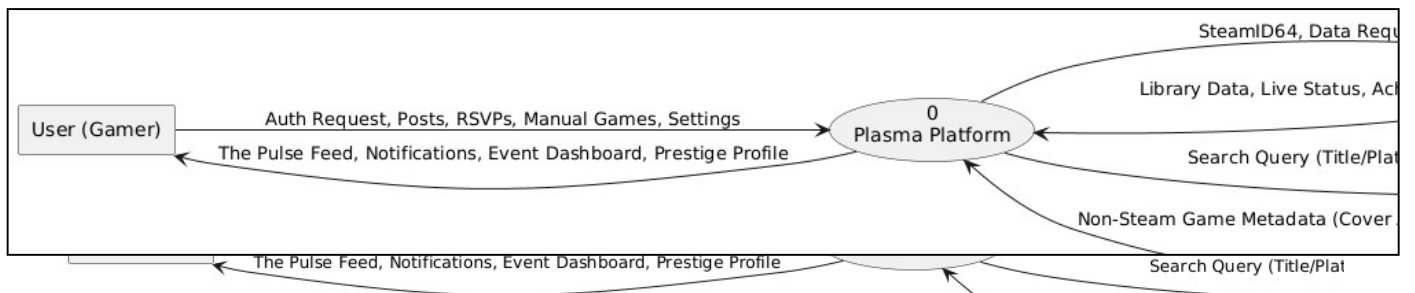
- **The Pulse (Social Feed):** An aggregated, real-time timeline that ingests Steam API status updates (e.g., “User started playing Elden Ring”) and merges them with user-generated posts, screenshots, and comments.
- **The Rally (Event Scheduling):** A timezone-aware calendar system allowing users to create multiplayer gaming events, assign specific squad roles (e.g., Healer, DPS), and track RSVPs.
- **The Omni-Library (Manual Tracking):** Virtual shelf utilizing the IGDB API to let users search, add, and broadcast their active status for non-Steam games.



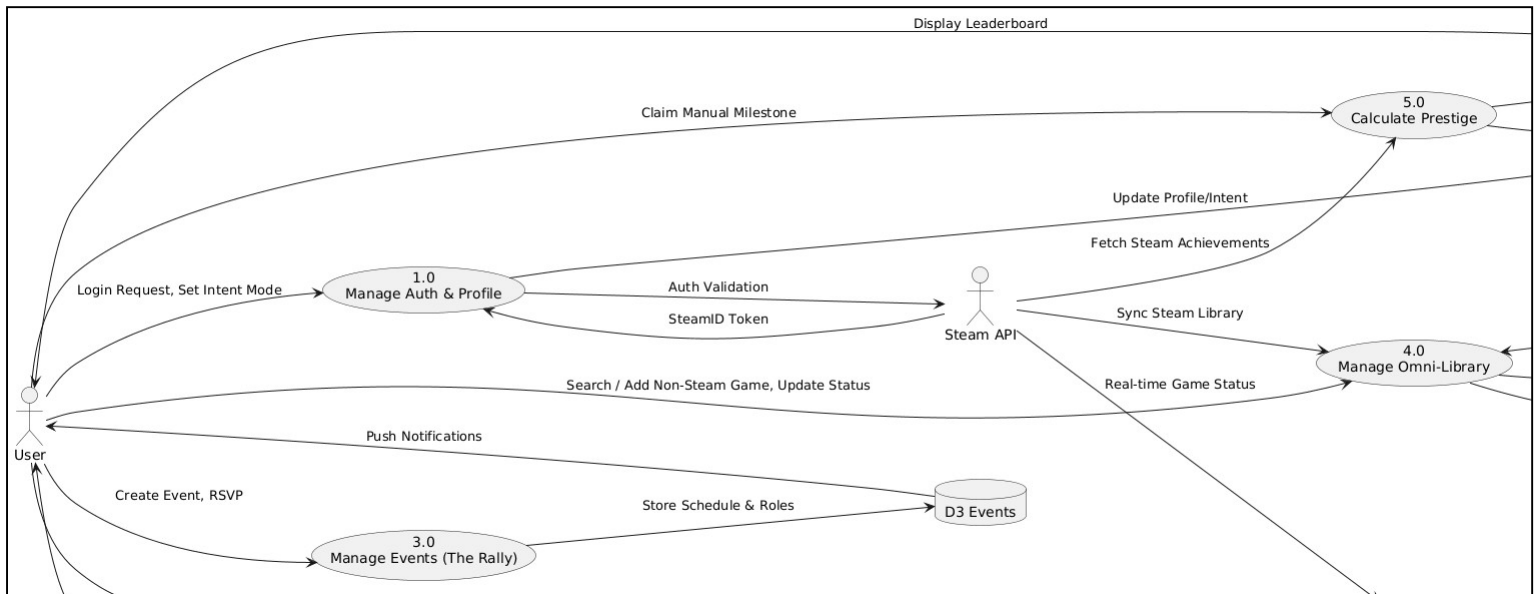
Use Case Diagram



Abstract Class Diagram



Level 0 DFD



Level 1 DFD

2.3 User Classes and Characteristics

We anticipate the platform will be utilized by four distinct user classes within the PC Gaming ecosystem, each with specific needs and usage patterns:

- **The Shotcallers (Squad Leaders):** The organizers of the friend group. They are the ones constantly asking “When are we playing?” and “Who is online?” They need **The Rally** to schedule sessions and **The Pulse** to see who is available without messaging five different people.
- **The Completionists (Achievement Hunters):** High-achievers who play games to 100% completion. They care about badges, rarity, and statistics. They use **The Prestige** to display their rare trophies and compare “Gamer Scores” on the leaderboard, which Steam does not natively offer in a competitive format.
- **The Variety Gamers (The Socialites):** Players who jump between trending games (e.g., Palworld, Helldivers) solely to hang out with friends. They use **Intent Modes** to signal they are just “Chilling” and rely on the feed to see what the group is playing this week.
- **The Pro-Am Aspirants (Competitive Players):** Players focused on climbing ranks in competitive shooters or MOBA titles. They use **Comp Mode** to find reliable teammates and filter out casual players who might hurt their rank.

2.4 Operating Environment

Plasma is a platform-agnostic web application. It will operate seamlessly across modern web browsers (Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge).

- **Desktop:** It will function as a browser-based dashboard on Windows, macOS, and Linux.

- **Mobile:** It must be fully responsive for iOS and Android browsers, utilizing Progressive Web App (PWA) standards to allow users to “Install to Home Screen” for an app-like experience.
- **Coexistence:** The software must peacefully coexist with local desktop environments by utilizing standard URL protocol handlers (e.g., `steam://run/<AppID>`) to hand off game-launching commands to the operating system’s native Steam client.

2.5 Design and Implementation Constraints

The development and design of Plasma are limited by several critical constraints:

- **API Rate Limiting:** The platform relies heavily on the Steam Web API, which imposes strict rate limits. The backend architecture must implement aggressive caching (e.g., Redis) to minimize redundant external API calls.
- **Authentication Standards:** Valve Corporation mandates the use of OpenID 2.0 for authenticating Steam identities. Developers cannot use modern OAuth 2.0 for Steam logins, constraining the auth architecture.
- **Read-Only Ecosystem:** The Steam API is strictly read-only for third-party apps. Plasma cannot alter a user’s Steam profile, delete their Steam friends, or unlock Steam achievements. All “write” actions (posts, RSVPs, manual games) are contained entirely within Plasma’s proprietary database.
- **Privacy Dependencies:** Plasma can only extract data from Steam accounts set to “Public.” If a user’s Steam profile is “Private,” the core functionalities of **The Pulse** and **The Prestige** will fail.

2.6 User Documentation

User documentation will be embedded directly into the platform to ensure a frictionless user experience:

- **Interactive Onboarding:** A guided, step-by-step tooltip overlay upon first login to explain how to link a Steam account and set up the **Omni-Library**.
- **Frequently Asked Questions (FAQ):** A dedicated web page explaining the **Double Opt-In** messaging rules, how **Plasma XP** is calculated, and troubleshooting steps for Steam privacy settings.

2.7 Assumptions and Dependencies

The successful delivery and operation of this project depend on several assumed factors and external dependencies:

- **Steam API Stability:** It is assumed that Valve Corporation will maintain the public availability of the Steam Web API without sudden deprecation or restrictive paywalls.
- **Global Game Database Access:** The Omni-Library feature depends on integrating a third-party, commercial database API, i.e. IGDB, to provide search results and metadata (box art, release dates) for non-Steam games. If this third-party service alters its pricing or access, the manual tracking feature will be severely impacted.
- **User Privacy Compliance:** We assume users are willing to make their Steam “Game Details” public in order to utilize the platform’s core tracking features.

3. External Interface Requirements

3.1 User Interfaces

Plasma will be delivered as a responsive Single Page Application (SPA) accessible via standard web browsers.

- **Design Standards:** The user interface will utilize a “Glassmorphism” or high-contrast dark mode aesthetic tailored for a gaming audience, ensuring it reduces eye strain during prolonged use.
- **Standard Layout Constraints:** Every screen will feature a persistent left-hand navigation menu (on desktop) or a bottom navigation bar (on mobile) containing quick links to the four core subsystems: **The Pulse**, **The Rally**, **The Omni-Library**, and **The Prestige**.
- **Deep Linking:** Game launch buttons across the UI will not execute local files; instead, they will trigger standard browser URI protocol handlers (e.g., `steam://run/<AppID>`) to prompt the user’s OS to open the respective game client.
- **Error Handling:** The UI will utilize standard, non-intrusive toast notifications for standard errors (e.g., “Failed to fetch live status” or “Event time conflicts with existing Rally”), maintaining standard error message display guidelines.

3.2 Hardware Interfaces

This section describes the logical and physical characteristics of each interface between the software product and the hardware components of the system.

- **Supported Device Types:** Plasma does not require or interact with any specialized physical hardware. It is a cloud-based web application that relies entirely on the standard hardware interfaces of the user’s host device (PC, Mac, or Smartphone), utilizing standard network interface cards for connectivity and standard displays for output.

3.3 Software Interfaces

There are the following connections between Plasma and other specific software components, including APIs and databases:

- **Steam Web API:**
 - **Purpose:** The primary external dependency for authenticating users, fetching live player summaries, retrieving owned game lists, and polling global achievement data.
 - **Data Items:** Incoming data consists of JSON payloads containing SteamID64s, AppIDs, and boolean status flags.
- **IGDB API:**
 - **Purpose:** Required by **The Omni-Library** to fetch accurate metadata for non-Steam games.
 - **Data Items:** Incoming data includes game titles, cover art URLs, and platform tags requested via search queries.
- **Primary Database (e.g., PostgreSQL):**
 - **Purpose:** The internal database storing the application’s proprietary data (user profiles, Intent Modes, social graphs, and Rally schedules).
 - **Data Sharing:** This database maps the external SteamID64 to a localized PlasmaUserID to bridge external API data with internal social interactions.

- **Operating System (OS) URI Protocols:**
 - **Purpose:** Plasma will interface with the host OS's application registry to hand off "Play" commands to installed software (e.g., the Steam Client).

3.4 Communications Interfaces

This section describes the requirements associated with any communications functions required by this product.

- **Network Protocols:** All client-to-server and server-to-API communication must be executed over secure HTTPS protocols to ensure data integrity and user privacy in transit.
- **Real-Time Data Transfer:** To populate the live activity on **The Pulse**, the application will utilize WebSockets or HTTP long-polling to maintain a persistent connection, fetching status updates from the backend aggregator service.
- **Web Push Notifications:** **The Rally** subsystem will utilize standard Service Workers and the Web Push API to send event reminders directly to the user's browser or mobile device, even when the application tab is closed.
- **Message Formatting:** All asynchronous communication between the Plasma frontend and backend APIs will be formatted strictly as JSON.

4. System Features

This section organizes the functional requirements of the Plasma platform by its major services and subsystems.

4.1 The Pulse (Unified Activity Feed)

4.1.1 Description and Priority

The Pulse is a High Priority feature. It serves as the core social aggregator that replaces the traditional storefront, dynamically pulling live status updates from the Steam API and merging them with user-generated moments into a single, scrollable timeline.

4.1.2 Stimulus/Response Sequences

- **Action:** A user loads the main dashboard.
 - **Response:** The system polls the Steam Web API for the live status of the user's friend network, retrieves recent database posts, and renders the unified feed.
- **Action:** A user clicks the "Join" button on a friend's active game post.
 - **Response:** The system triggers the steam:// URI protocol to launch the game client locally and attempt a lobby connection.

4.1.3 Functional Requirements

The following itemizes the detailed functional requirements associated with this feature:

- **REQ-1:** The system shall authenticate and fetch the user's friend list and live gaming status via the Steam Web API using the user's linked SteamID64.

- **REQ-2:** The system shall allow users to publish text, image, and video clip “Moments” directly to their followers’ Pulse feed.
- **REQ-3:** The system shall execute OS-level application deep links when a user interacts with a “Play” or “Join” call-to-action on a feed item.

4.2 The Rally (Event Coordination Engine)

4.2.1 Description and Priority

The Rally is a High Priority feature. It is a visual, calendar-based scheduling tool designed to eliminate timezone confusion and role allocation friction for multiplayer gaming sessions.

4.2.2 Stimulus/Response Sequences

- **Action:** A *Shotcaller* user creates a **Rally** event with a specific UTC timestamp.
 - **Response:** The system saves the event and displays it to all invitees, automatically converting the start time to each viewing user’s local timezone.
- **Action:** The system clock reaches 15 minutes prior to a scheduled **Rally**.
 - **Response:** The system dispatches a browser push notification to all users who have RSVP’d to the event.

4.2.3 Functional Requirements

The following itemizes the detailed functional requirements associated with this feature:

- **REQ-4:** The system shall dynamically calculate and display event times based on the viewing user’s local browser timezone settings.
- **REQ-5:** The system shall allow users to RSVP to an event by selecting predefined or custom in-game roles (e.g., DPS, Healer, Tank).
- **REQ-6:** The system shall utilize Service Workers to deliver web push notifications 15 minutes prior to a **Rally’s** start time.

4.3 The Omni-Library (Manual Game Tracking)

4.3.1 Description and Priority

The Omni-Library is a Medium Priority feature. It is a database-driven subsystem allowing users to manually search for, add, and broadcast their status for non-Steam games (e.g., Riot Games, Epic Games, or Blizzard titles).

4.3.2 Stimulus/Response Sequences

- **Action:** A user searches for a game title not available on Steam.
 - **Response:** The system queries a global game database API (e.g., IGDB) and returns the corresponding metadata (box art, platform, release date) for the user to add to their profile.
- **Action:** A user manually toggles their status to “Playing [Non-Steam Game]”.
 - **Response:** The system updates the internal database and broadcasts this active status to The Pulse for their network to see.

4.3.3 Functional Requirements

The following itemizes the detailed functional requirements associated with this feature:

- **REQ-7:** The system shall integrate with a third-party game database API to retrieve accurate metadata for non-Steam titles.
- **REQ-8:** The system shall allow users to manually toggle an active “Playing” status for games in their Omni-Library, mirroring the automatic Steam status behavior on the social feed.

4.4 The Prestige (Meta-Gamification)

4.4.1 Description and Priority

The Prestige is a Medium Priority feature. It is a universal progression system that tracks Steam achievements, calculates rarity-weighted scores, and generates global leaderboards to gamify the completionist experience.

4.4.2 Stimulus/Response Sequences

- **Action:** A user navigates to a peer’s **Prestige** profile.
 - **Response:** The system fetches the peer’s raw Steam achievements, calculates the global rarity weights for each, and renders the total accumulated **Plasma XP** and Hall of Fame.

4.4.3 Functional Requirements

The following itemizes the detailed functional requirements associated with this feature:

- **REQ-9:** The system shall assign point values (**Plasma XP**) to unlocked achievements inversely proportional to their global unlock percentage.
- **REQ-10:** The system shall allow users to pin up to five specific achievements to a visual “Hall of Fame” on their public profile.
- **REQ-11:** The system shall allow users to manually claim major milestones for **Omni-Library** games, subject to the platform’s verification rules.

4.5 Intent Modes (Social Signaling)

4.5.1 Description and Priority

Intent Modes are a Medium Priority feature. This subsystem allows users to set a global status flag (e.g., *Comp Mode*, *Chill Mode*) that dictates their current social availability and gaming mindset, reducing social friction and helping players find appropriate squadmates.

4.5.2 Stimulus/Response Sequences

- **Action:** A user toggles their **Intent Mode** from the main navigation bar.
 - **Response:** The system immediately updates the user’s active status in the database, alters the visual border/badge on their profile picture, and broadcasts a status change to **The Pulse**.

- **Action:** A user applies an *Intent Filter* to **The Rally** event board.
 - **Response:** The system queries the database and hides all scheduled events that do not match the selected intent (e.g., hiding casual custom games when filtering for *Comp Mode*).

4.5.3 Functional Requirements

The following itemizes the detailed functional requirements associated with this feature:

- **REQ-12:** The system shall allow users to toggle their global social intent state between defined modes (e.g., *Competitive*, *Casual/Chill*, *Offline*).
- **REQ-13:** The system shall visually distinguish a user's active **Intent Mode** across all UI surfaces, including their profile card, **Rally** RSVPs, and **Pulse** feed posts.
- **REQ-14:** The system shall provide filtering mechanisms on **The Pulse** and **The Rally** to allow users to view content and events strictly matching a specific **Intent Mode**.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

This section defines the performance constraints the platform must operate under to ensure a seamless user experience and prevent API throttling.

- **Feed Rendering:** The system must render **The Pulse** feed within 1.5 seconds of a user loading the dashboard under normal load conditions.
- **API Rate Limiting:** Because Plasma relies heavily on external data, the backend must implement a robust caching layer (e.g., Redis). Polling the Steam Web API and IGDB API must be throttled and cached to strictly adhere to their respective rate limits, preventing IP blacklisting.
- **Notification Latency:** Push notifications for **The Rally** must be dispatched asynchronously and arrive within a 1-minute variance of the 15-minute pre-event threshold.

5.2 Safety Requirements

While Plasma is a digital social platform without direct risk of physical harm, specific safeguards must be taken to prevent data loss and system-level disruption.

- **Data Integrity:** Automated, geo-redundant daily backups of the primary PostgreSQL database must be implemented to prevent the loss of user social graphs and **Omni-Library** configurations.
- **Protocol Handoff Safety:** The UI must clearly telegraph when a user is about to launch an external application. Triggering steam:// URI protocols must not execute without explicit user interaction (e.g., clicking "Play") to prevent unexpected operating system behavior or background application launches.

5.3 Security Requirements

This section outlines the requirements for protecting user identity and data integrity.

- **Authentication:** Plasma shall not request or store user passwords. All primary identity authentication must be securely handled via Valve's OpenID 2.0 protocol.
- **Encryption:** All data transmitted between the client browser, the Plasma backend, the Steam Web API, and the IGDB API must be encrypted using HTTPS/TLS 1.2 or higher.
- **Vulnerability Mitigation:** The web application must implement modern security headers and standard protections against Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and SQL injection.

5.4 Software Quality Attributes

The following quality characteristics are critical to the success and adoption of the Plasma platform:

- **Availability:** The platform must act as a reliable "Third Place" for gamers. The infrastructure should target 99.9% uptime, utilizing auto-scaling containerized microservices to handle traffic spikes during major game releases or peak weekend hours.
- **Usability:** The interface must be highly intuitive, heavily prioritizing ease of use over ease of learning. It must seamlessly adapt its layout from 4K desktop monitors to mobile device screens without losing functionality.
- **Scalability:** The system's relational database must be optimized with proper indexing to ensure that querying the social graph (fetching posts from hundreds of followed users) remains performant as the user base grows.

5.5 Business Rules

The following operating principles govern user interactions within the application ecosystem:

- **BR-1 (Account Verification):** Access to Plasma is strictly gated; a user must successfully link a valid Steam account via OpenID to create a profile and access the platform.
- **BR-2 (Anti-Harassment):** Direct messaging and external lobby invites are strictly governed by a **Double Opt-In** rule. Users cannot interact privately unless they mutually follow each other or are actively RSVP'd to the same **Rally** event.
- **BR-3 (Milestone Integrity):** Manual achievement claims for **Omni-Library** games (**The Prestige**) must be accompanied by user-uploaded proof (e.g., a screenshot).

6. Other Requirements

This section defines additional requirements critical to the system's operation that are not covered in the functional or nonfunctional sections.

- **Primary Relational Store:** The system shall utilize PostgreSQL to manage highly relational data, including the user social graph (followers/following), **Rally** event rosters, and **Omni-Library** configurations.
- **In-Memory Caching:** The system shall implement a Redis caching layer to store transient data, specifically the frequently polled live status updates from the Steam Web API and search results from the IGDB API, to drastically reduce latency and prevent API rate limiting.

- **Timezone Normalization:** As established in Section 4.2, all backend timestamps must be stored in Coordinated Universal Time (UTC) and dynamically localized to the user's browser timezone on the frontend client.
- **Data Processing Consent:** Upon initial Steam OpenID authentication, users must be presented with a clear Terms of Service and Privacy Policy outlining what data Plasma fetches from Steam and IGDB.
- **Right to be Forgotten (GDPR/CCPA Compliance):** The system must provide a frictionless, one-click "Delete Account" function in the user settings. Executing this must permanently scrub the user's SteamID64, social connections, posts, and **Rally** history from the primary PostgreSQL database within 30 days.

Appendix A: Glossary

This section defines terms, acronyms, and abbreviations necessary to interpret this SRS.

- **Deep Link / URI Protocol:** An operating system-level command that allows a web browser to launch a local desktop application. Plasma uses `steam://run/<AppID>` to launch games.
- **Double Opt-In:** A security and anti-harassment protocol where two users must mutually agree to a connection (e.g., mutually following each other) before direct messaging or lobby invites are permitted.
- **IGDB:** Internet Game Database. A comprehensive, third-party video game database API (operated by Twitch) used by Plasma to fetch metadata for non-Steam games.
- **Omni-Library:** Plasma's manual game tracking subsystem that allows users to display and update their status for games outside the Steam ecosystem.
- **OpenID 2.0:** A decentralized authentication protocol. Valve Corporation uses this specifically for secure third-party Steam logins without exposing user passwords.
- **Plasma XP:** The proprietary, gamified point system calculated by **The Prestige** subsystem, weighted by the global rarity of a user's unlocked achievements.
- **SteamID64:** A unique, 17-digit numeric identifier assigned to every Steam account, used as the primary key for all Steam Web API data requests.
- **The Pulse:** The core social timeline and feed subsystem of the Plasma platform.
- **The Rally:** The calendar-based event scheduling subsystem.

Appendix B: To Be Determined List

The following numbered items represent technical or business logic references that require further research or stakeholder resolution before final implementation:

- **TBD-1:** The exact optimal background polling frequency for the Steam Web API to guarantee live status accuracy on **The Pulse** without triggering Valve's rate-limit blacklisting.
- **TBD-2:** The specific caching Time-To-Live (TTL) duration for IGDB metadata to balance data freshness against the API's monthly query quota.
- **TBD-3:** The precise mathematical algorithm used by **The Prestige** to calculate the rarity weight of **Plasma XP** for older, legacy Steam games versus newly released titles.
- **TBD-4:** The specific cloud hosting provider (e.g., AWS, Azure, Vercel) and exact hardware specifications for the production database servers.